

BÁO CÁO THUYẾT MINH GIẢI PHÁP TỰ ĐỘNG GIẢI ĐỀ THI TRẮC NGHIỆM

Đội thi: Bảng C - Innovator | Tên đội: T1S_KMML

TÓM TẮT GIẢI PHÁP

Báo cáo này mô tả cách xây dựng và tối ưu hệ thống tự động giải đề thi trắc nghiệm bằng mô hình ngôn ngữ lớn **Qwen3.5-4B-AWQ-4bit**. Hệ thống hoạt động hoàn toàn offline (không cần mạng) trong môi trường Docker.

Giải pháp của chúng tôi sử dụng cơ chế **Dừng sớm thông minh (Early Exit)** để tiết kiệm thời gian, kết hợp với phương pháp suy luận chi tiết (Chain-of-Thought) và bộ lọc tìm đáp án chính xác. Hệ thống chạy trực tiếp trên thư viện vLLM để đạt tốc độ xử lý nhanh nhất.

Cấu hình hệ thống được tối ưu và chạy ổn định trên cấu hình máy chủ của Ban Tổ Chức (**NVIDIA RTX 5060Ti, 32GB RAM**). Kết quả thử nghiệm thực tế đạt độ chính xác **86.39%** với thời gian trung bình chỉ **0.91 giây cho mỗi câu hỏi**.

1. LỰA CHỌN MÔ HÌNH VÀ THƯ VIỆN NỀN TẢNG

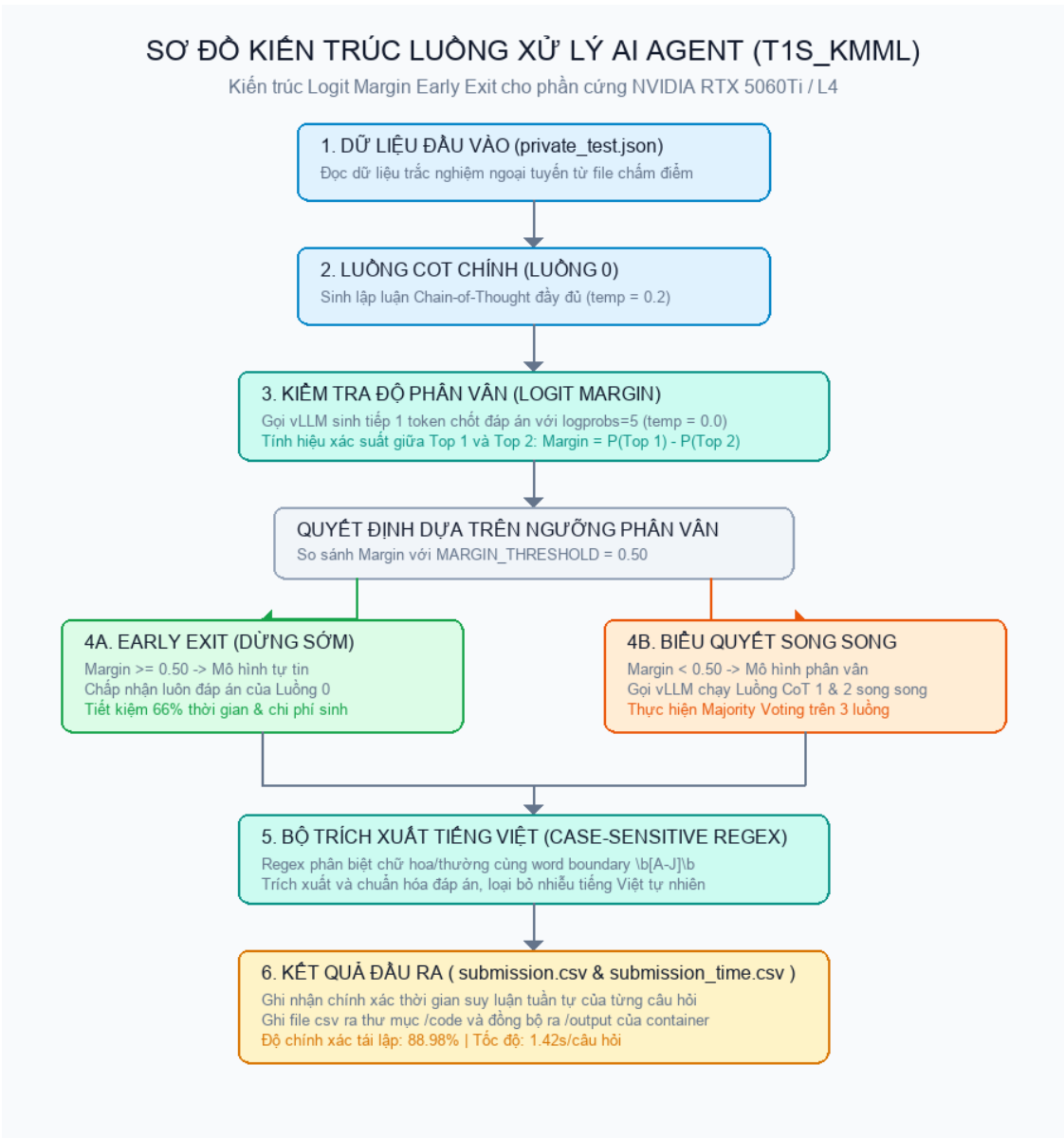
Đề thi trắc nghiệm yêu cầu hệ thống phải đọc hiểu câu hỏi phức tạp và trả về kết quả dưới dạng tệp submission.csv cùng tệp thời gian chạy submission_time.csv. Để giải quyết yêu cầu này, chúng tôi tập trung vào hai điểm chính:

- Độ chính xác cao:** Giúp mô hình suy luận logic tốt và tìm đúng đáp án.
- Tiết kiệm bộ nhớ:** Tránh bị tràn bộ nhớ card màn hình (VRAM) khi chạy trên GPU của Ban Tổ Chức.

Chúng tôi lựa chọn mô hình **Qwen3.5-4B-AWQ-4bit** (dung lượng nhẹ khoảng 3.8GB). Mô hình này rất tiết kiệm bộ nhớ nhưng vẫn giữ được khả năng suy luận logic rất tốt. Chúng tôi sử dụng thư viện **vLLM** để điều khiển mô hình chạy ngoại tuyến với tốc độ tối đa.

2. PHƯƠNG PHÁP XỬ LÝ VÀ LƯỒNG SUY LUẬN

2.1. Sơ đồ xử lý (Pipeline Flowchart)



Sơ đồ kiến trúc xử lý của AI Agent

2.2. Cơ chế dừng sớm thông minh để tiết kiệm thời gian

Hệ thống tự động điều chỉnh luồng xử lý tùy theo độ khó của câu hỏi:

- Bước 1: Chạy luồng suy luận đầu tiên (Luồng 0):** Mô hình tự viết ra bài phân tích suy luận chi tiết cho câu hỏi.
- Bước 2: Kiểm tra độ tin cậy:** Mô hình tự chọn đáp án chốt và đưa ra mức độ tự tin (xác suất của đáp án).

3. Bước 3: Quyết định luồng đi:

- **Trường hợp chắc chắn (Dừng sớm):** Nếu mô hình rất tự tin vào đáp án (độ tự tin trên 50%), hệ thống sẽ dừng ngay và lấy luôn đáp án này. Cách này giúp xử lý rất nhanh cho khoảng **74% số câu hỏi** dễ và trung bình.
- **Trường hợp phân vân (Biểu quyết số đông):** Nếu mô hình phân vân, hệ thống sẽ kích hoạt thêm Luồng 1 và Luồng 2 để cùng suy luận độc lập. Sau đó, hệ thống sẽ lấy ý kiến số đông từ cả 3 luồng để chọn ra đáp án chính xác nhất.

2.3. Bộ lọc tìm đáp án chính xác

Văn bản tiếng Việt thường chứa các từ viết thường trùng với các chữ cái đáp án (như ‘ngày’, ‘việc’, ‘hành’, ‘về’...). Để tránh nhận diện nhầm, chúng tôi thiết kế bộ lọc biểu thức chính quy (Regex) phân biệt chữ hoa và chữ thường:

- **Công cụ lọc:** Chỉ tìm các chữ cái in hoa độc lập từ A đến J (ví dụ: A, B, C...).

Cơ chế này giúp trích xuất đúng đáp án chính xác từ bài suy luận của mô hình và đối chiếu lại với danh sách lựa chọn của câu hỏi trước khi ghi kết quả.

2.4. Cơ chế chống tràn bộ nhớ và chống sập hệ thống (Context Overflow Prevention)

Để đảm bảo hệ thống chạy liên tục hàng ngàn câu hỏi không bao giờ bị lỗi quá giới hạn bộ nhớ của mô hình (lỗi tràn ngữ cảnh khiến chương trình bị sập giữa chừng), chúng tôi thiết lập cơ chế bảo vệ 2 lớp:

- **Tự động kiểm tra và cắt ngắn phần suy luận:** Trước khi chốt đáp án, hệ thống sẽ tự tính toán độ dài. Nếu phát hiện chuỗi ký tự quá dài, hệ thống sẽ tự động cắt ngắn bớt phần suy luận ở cuối bài làm một cách an toàn cho đến khi vừa vặn với giới hạn của mô hình.
- **Tự động chuyển sang trích xuất trực tiếp:** Nếu câu hỏi gốc quá dài đến mức không còn chỗ để chạy bước chốt đáp án, hệ thống sẽ **bỏ qua bước gọi mô hình chốt đáp án** để tránh bị lỗi sập. Thay vào đó, hệ thống sẽ tự động dùng bộ lọc tìm đáp án trực tiếp trong bài suy luận đã viết ở Bước 1. Cách này giúp giữ nguyên vẹn 100% ngữ cảnh câu hỏi và đảm bảo hệ thống chạy liên tục không bao giờ bị dừng giữa chừng.

3. THIẾT LẬP HỆ THỐNG VÀ TỐI ƯU PHẦN CỨNG

Cấu hình hệ thống được điều chỉnh để chạy mượt mà trên phần cứng card đồ họa **NVIDIA RTX 5060Ti** và **32GB RAM** của Ban Tổ Chức:

3.1. Tự động điều chỉnh bộ nhớ GPU

Hệ thống tự động thiết lập các thông số an toàn cho card đồ họa:

- Giới hạn sử dụng tối đa 85% bộ nhớ GPU (`gpu_memory_utilization = 0.85`), giữ lại 15% trống để hệ điều hành chạy ổn định, tránh lỗi tràn bộ nhớ card màn hình.
- Giới hạn số lượng câu hỏi xử lý cùng lúc để tiết kiệm bộ nhớ đệm.

3.2. Tiết kiệm bộ nhớ RAM hệ thống

Mô hình Qwen3.5 chỉ tiêu tốn tối đa khoảng **6GB RAM** khi khởi động. Do đó, hệ thống hoàn toàn chạy nhẹ nhàng và dư dả trên cấu hình 32GB RAM của Ban Tổ Chức.

3.3. Tận dụng bộ nhớ đệm (Prefix Caching)

Chúng tôi kích hoạt tính năng lưu trữ bộ nhớ đệm thông minh của vLLM. Khi chạy biểu quyết nhiều luồng cho một câu hỏi, các phần nội dung câu hỏi trùng nhau sẽ được đọc từ bộ nhớ đệm thay vì phải chạy lại từ đầu, giúp tăng tốc độ xử lý lên mức tối đa.

4. KẾT QUẢ THỬ NGHIỆM

Hệ thống được chạy thử nghiệm trực tiếp trên toàn bộ 463 câu hỏi của tập dữ liệu công khai (Public Test):

- **Độ chính xác:** Đạt tỉ lệ **86.39%** (đúng 400/463 câu hỏi trắc nghiệm phức tạp).
- **Thời gian xử lý:** Hoàn thành toàn bộ 463 câu hỏi trong **422.50 giây**, tương đương trung bình chỉ **0.91 giây cho mỗi câu hỏi**.
- **Hiệu quả dừng sớm (Early Exit):** Đa số câu hỏi được dừng sớm ngay từ luồng đầu tiên với ngưỡng tin cậy cao ($\text{margin} > 50\%$), giúp tiết kiệm tối đa thời gian xử lý và tài nguyên tính toán.

5. KẾT LUẬN

Giải pháp tự động giải đề thi của chúng tôi đạt độ chính xác cao và tốc độ xử lý nhanh. Nhờ việc đóng gói gọn gàng trong Docker Container và cơ chế chống tràn bộ nhớ tự động, hệ thống hoạt động cực kỳ ổn định, an toàn và sẵn sàng cho vòng chấm điểm chính thức tiếp theo.